

Лабораторная работа: Политики безопасности и аудит в Kubernetes

Цель работы: Изучить механизмы безопасности Kubernetes на практике: от эскалации привилегий через сервис-аккаунты до внедрения политик валидации и мутации с помощью Kyverno, а также освоить инструменты аудита соответствия CIS Benchmark.

Требования: Кластер Kubernetes (minikube, kind, или облачный), `kubectl`, `helm`, `kyverno`, `kube-bench`, `kubescape`.

Подготовка среды

1. Выполните установку кластера через minikube и запустите через `minikube start`

Перед началом работы убедитесь, что кластер доступен:

```
kubectl cluster-info
```

Создайте два пространства имен для экспериментов:

```
kubectl create namespace target-ns  
kubectl create namespace sensitive-ns
```

Задание 1. Эскалация привилегий: перехват секретов через сервис-аккаунт

Цель: Продемонстрировать риски, связанные с сервис-аккаунтами, имеющими избыточные права (`cluster-admin`), и показать, как злоумышленник может использовать их для доступа к секретам в других пространствах имен.

1.1 Развертывание демо-приложения с широкими правами

Используйте Helm-чарт из `application.zip`. Установите приложение:

```
helm install demo-attack ./application --namespace <target-ns> --create-namespace
```

Убедитесь, что под запущен и использует сервис-аккаунт `wide-sa`:

```
kubectl get pods -n target-ns  
kubectl describe pod -n target-ns -l app.kubernetes.io/instance=demo-attack | grep  
ServiceAccount
```

1.2 Создание тестового секрета в другом пространстве имен

Создайте секрет в пространстве `sensitive-ns`, который будет целью "атаки"

1.3 Атака: получение секретов через под с широкими правами

1. Получите имя пода из
2. Получите содержимое сервисного аккаунта, примонтированного к поду и внутри контейнера выполните запрос к Kubernetes API

Ожидаемый результат: Вы успешно получите список всех секретов из пространства `sensitive-ns`, включая `db-credentials`, так как сервис-аккаунт имеет права `cluster-admin`.

1.4 Удаление

```
helm uninstall demo-attack -n target-ns
kubectl delete secret db-credentials -n sensitive-ns
```

Задание 2. Валидирующие политики Kyverno: запрет на Pod Security

Цель: Установить Kyverno и создать политику валидации, которая запрещает использование привилегированных контейнеров, с применением условия на определенный ресурс.

2.1 Установка Kyverno

Установите Kyverno в кластер с помощью Helm:

```
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update
helm install kyverno kyverno/kyverno --namespace kyverno --create-namespace
```

Проверьте, что контроллеры Kyverno запущены:

```
kubectl get pods -n kyverno
```

2.2 Создайте валидирующую политику, блокирующую 1 любой параметр безопасности из PSP. Для примера можно воспользоваться банком политик Kyverno, объяснить опасность используемого параметра, продемонстрировать блокировку ресурса (можно использовать предоставленный helm chart). В политику добавить precondition (сработка на ресурсы только с определенным паттерном в названии, например). Можно использовать как ValidationPolicy, так и ClusterPolicy (kyverno версии 1.14 поддерживает)

Задание 3. Мутирующие политики Kyverno: мутирование секретов

Цель: Создать мутирующую политику, которая убирает у роли возможность читать секрет. По примеру политик и версии киверно аналогично пункту выше

Задание 4. Аудит безопасности: kube-bench и Kubescape

Цель: Выполнить сканирование кластера с помощью `kube-bench` (безопасность нод) и `kubescape` (безопасность манифестов), интерпретировать результаты.

4.1 Запуск kube-bench (CIS Benchmark для нод)

`kube-bench` проверяет настройки узлов (master/worker) по стандартам CIS. Если вы используете `minikube` или `kind`, некоторые проверки могут быть пропущены, но принцип работы остается.

Установка и запуск:

Репозиторий: <https://github.com/aquasecurity/kube-bench>

Задание: Найдите в отчете любую проверку со статусом `FAIL`, опишите чем использование этого параметра небезопасно

4.2 Запуск Kubescape (безопасность манифестов и cluster configuration)

`Kubescape` сканирует манифесты (или live-кластер) на соответствие NSA, MITRE ATT&CK и CIS.

Репозиторий: <https://github.com/kubescape/kubescape>

Интерпретация результатов: На основе вывода определите:

1. Сколько критических ресурсов (например, `cluster-admin`) обнаружено в кластере.
2. Есть ли контейнеры, работающие с `privileged: true`.
3. Рекомендации по исправлению для наиболее критичного FAIL.